

DSDM3 - Vignette

Introduction

In this vignette, we demonstrate how to implement the discrete sparse Dirichlet-multinomial mixture model (DSDM³) presented in “A Bayesian Semiparametric Mixture Model for Clustering Zero-Inflated Microbiome Data” by S Korsurat and MD Koslovsky. We begin by providing instructions for simulating the data used in the simulation study of the main manuscript. We then illustrate how to apply the model to both the simulated data and the Enteric Diarrheal Diseases (EDD) data featured in the application study.

Required Libraries

To implement our package, the following R packages must be installed and loaded into your R environment:

- devtools
- Rcpp
- RcppArmadillo
- tidyverse
- ggplot2
- dirmult
- salso
- mclustcomp

If any of these packages are not already installed, you can install them using the `install.packages()` function. For example:

```
install.packages( "tidyverse" )
```

After installing the required packages, use the following command to load our package into the R environment:

```
library( DSDM3 )
```

Simulate Data

In this section, we demonstrate how to simulate data similar to that used in the main manuscript using the `sim_ZIDMclus()` function, which generates two clusters of zero-inflated Dirichlet-multinomial distributed data.

```
### Simulate the data
simdat_ZIDM <- sim_ZIDMclus( n1 = 25, n2 = 25, Jnoise = 80, Jsignal = 20,
                             z_sum_noise = 4000, z_sum_signal = 1000,
                             a = 25, pAR0 = 0.75, seed = 10 )
```

The `sim_ZIDMclus()` function requires several parameters: the number of observations in each cluster (`n1` and `n2`), the number of taxa that are not differentiated across clusters, or the noise taxa, (`Jnoise`), the number of taxa that are differentiated between clusters, or the signal taxa, (`Jsignal`), the sequencing depths for noise and signal taxa (`z_sum_noise` and `z_sum_signal`, respectively), a random seed (`seed`), and two parameters, `a` and `pAR0`, which control the behavior of at-risk and structural zeros. Specifically, `pAR0` defines the baseline probability that a zero count is considered at-risk when the expected relative abundance is zero, while `a` controls how this probability changes with abundance—smaller values (i.e., $0 < a < 1$) make low-abundance taxa more likely to be treated as at-risk, whereas larger values (i.e., $a > 1$) make them more likely to be treated as structural zeros. In this example, we simulate 50 observations—25 for each cluster—with a total of 100 taxa, 20 of which are designated as signal taxa. The sequencing depth for each observation is set to 5,000 reads, allocated as 4,000 reads for the noise taxa and 1,000 reads for the signal taxa. The parameter `pAR0 = 0.75` specifies that when the expected marginal relative abundance of a taxon is zero, the probability of each zero count for that taxon being considered at-risk is 0.75. Additionally, with `a = 25`, zeros are more likely to be treated as structural when the expected marginal relative abundance is high.

The output from the `sim_clusDat()` function is a list consisting of three elements: (1) the simulated OTU table (`dat`), (2) the at-risk indicator (`ar`), and (3) the cluster allocation for each observation (`clus`). To visualize the simulated data (i.e., the OTU table), we can use the `vizDat()` function to create a heatmap that highlights patterns in the data. In the resulting plot, each row corresponds to an observation and each column to a taxon. The intensity of the color indicates the magnitude of the observed count—darker colors represent higher counts.

```
### Visualize the data
heatmap_ZIDM <- vizDat( otuTab = simdat_ZIDM$dat )
```

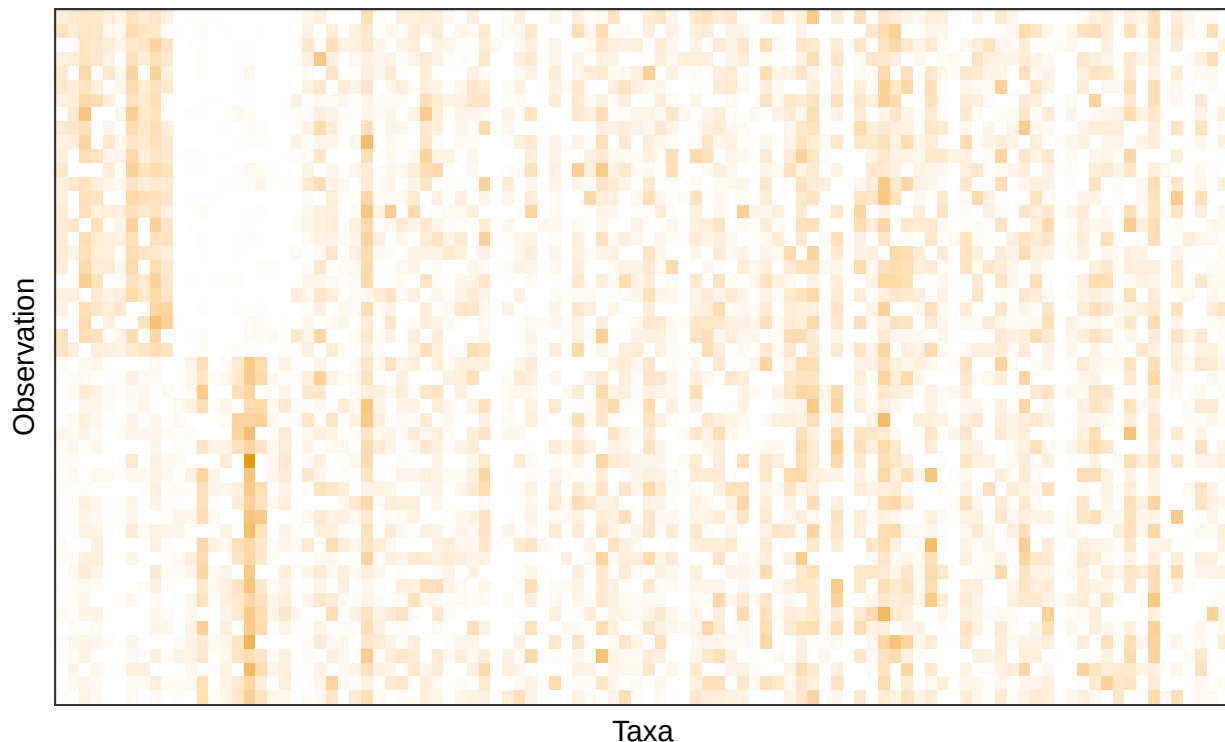


Figure 1: Heatmap for the simulated data

Implementation

In this section, we demonstrate how to implement DSDM³ on the simulated data using the `DSDM3_ZIDM()` function.

```
### Apply DSDM3 to the simulated data
result_ZIDM <- DSDM3_ZIDM( otuTab = simdat_ZIDM$dat, Km = 10, pi_TB = 0.5,
                           s2_prior = 0.1, s2_MH = 0.01, theta = 0.1,
                           a_gamma = 1, b_gamma = 1, Kp_init = 5,
                           s = 500, iter = 10000, thin = 1, seed = 10 )
```

This function requires the following inputs: the $N \times J$ OTU table (`otuTab`), where each row represents an observation and each column represents a taxon, the number of potential components (`Km`), the prior probability that a given component exists (`pi_TB`), the prior variance for the cluster concentration parameter (`s2_prior`), the variance used in the Metropolis–Hastings (MH) step when updating the cluster concentration parameter (`s2_MH`), the sparsity concentration parameter (`theta`), the parameters controlling the expected probability of being an at-risk zero (`a_gamma` and `b_gamma`), the initial number of clusters, i.e., non-empty components (`Kp_init`), the hypothetical total number of reads used to inform the prior (`s`), the number of MCMC iterations (`iter`), the thinning interval (`thin`), and the random seed (`seed`). In this example, we apply the model to the simulated data provided above (`simdat_ZIDM$dat`). The model is run for 10,000 iterations with no thinning (i.e., `thin = 1`). We set the prior variance of the cluster concentration parameter to 0.1 and the variance for the Metropolis–Hastings step to 0.01. The maximum number of components is limited to 10 (i.e., `Km = 10`). For at-risk zeros, we assume the expected probability of a zero being considered at-risk is 0.5 (i.e., `a_gamma = b_gamma = 1`). We also assume 500 hypothetical total reads are used to inform the prior (i.e., `s = 500`). Additionally, we initialize the model with five random clusters (i.e., `Kp_init = 5`).

The output from the `DSDM3_ZIDM()` function is a list object. To obtain the final cluster assignment, we use the `final_clus()` function. For implementation, we need to specify the number of iterations to consider as burn-in. We utilize the `salso()` function from the `salso` package with the variation of information loss to determine the final cluster assignment (Dahl, Johnson, and Müller 2022).

```
### Obtain a vector of the final cluster assignment.
clusResult <- final_clus( resultList = result_ZIDM, burn_in = 1000, seed = 10 )
```

Posterior Inference

Prior to performing inference on the cluster allocation, we assess model convergence by plotting the number of active clusters and the component weights across MCMC iterations, as suggested by Frühwirth-Schnatter, Malsiner-Walli, and Grün (2021). The `diag_plot()` function produces plots for both the number of active clusters and the component weights, using the resulting list returned by our model. As shown in Figure 2 and 3, the model appears to converge around 1,000th iteration.

```
### Obtain the diagnostic plots
diagplot_ZIDM <- diag_plot( result_ZIDM )
```

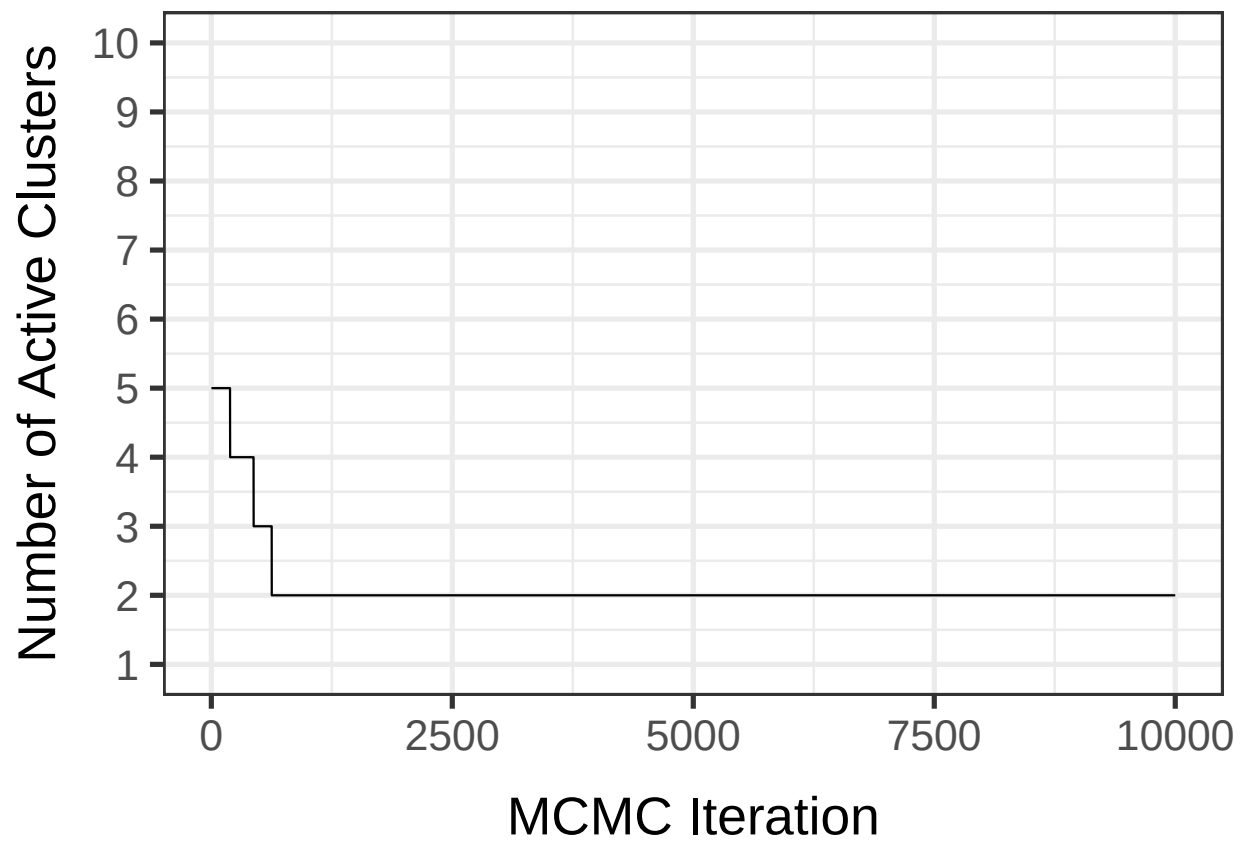


Figure 2: Simulation Results: Trace plot for the number of active clusters in each MCMC iteration when the model is applied to the simulated data.

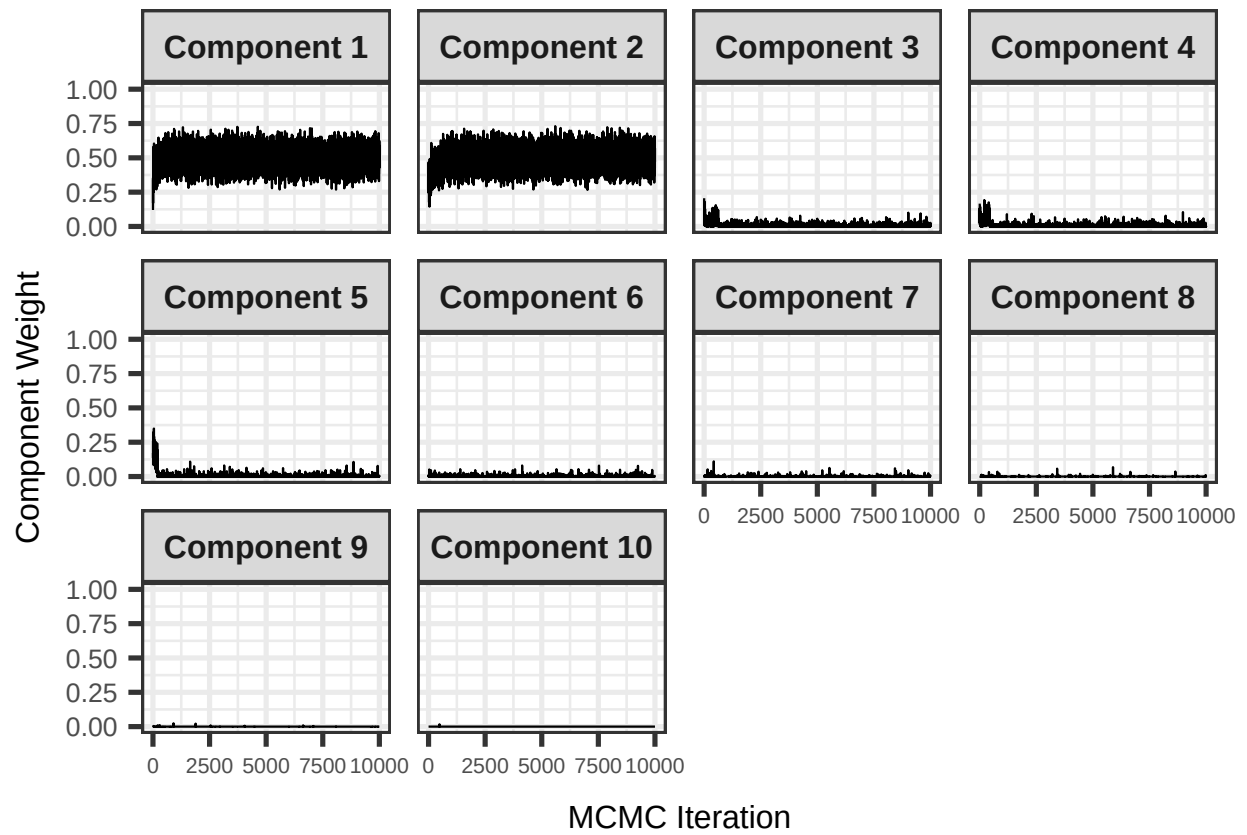


Figure 3: Simulation Results: Trace plot for the component weight in each MCMC iteration when the model is applied to the simulated data.

Next, we assess the performance of the resulting cluster allocation compared to the true cluster assignment (if applicable) using the Adjusted Rand Index (ARI) with the `ARI_clus()` function. ARI measures the similarity between two cluster assignment vectors, ranging from -1 to 1. The higher the ARI, the more similar the result to the truth. ARI below 0 indicates that the clustering result is worse than what would be expected by random chance.

```
### Calculate the ARI
ARI_clus( resultList = result_ZIDM, truth = simdat_ZIDM$clus, burn_in = 1000, seed = 1 )
#> [1] 1
```

We observe that the ARI equals 1, demonstrating that our model successfully differentiated between the two clusters.

Application Data

In this section, we apply the proposed model to the EDD dataset analyzed in the main manuscript, originally collected by Singh et al. (2015) and made publicly available by Duvallet et al. (2017). We begin by loading the data into the R environment. The dataset is stored as a list object with three components: (1) `metadata`, which indicates whether each sample is from an EDD patient or a healthy control, (2) `genus`, the OTU table at the genus level after data cleaning, and (3) `de_novo`, the OTU table at the de novo level after data cleaning. In the main manuscript, we use the OTU table at the genus level, resulting in a 303×79 -dimensional OTU table for inference.

```
### Import the dataset
data( "dat_EDD" )
```

We then apply the proposed model to the genus-level OTU table, running the MCMC sampler for 15,000 iterations with thinning every 2 iterations. The variance of the cluster concentration parameter is set to 10, and the variance for the MH step is set to 1. The model is limited to a maximum of 10 components. For at-risk zeros, we assume the expected probability of a zero being considered at-risk is 0.5. For initialization, the model begins with a single cluster containing all samples, and we assume 200 hypothetical total reads are used to inform the prior.

```
### Apply the model to the genus-level OTU table.
result_EDD <- DSDM3_ZIDM( otuTab = dat_EDD$genus, Km = 10, pi_TB = 0.5,
                          s2_prior = 10, s2_MH = 1, theta = 0.1,
                          a_gamma = 1, b_gamma = 1, Kp_init = 1,
                          s = 200, iter = 10000, thin = 2, seed = 1 )
```

We obtain the final cluster assignment by using `final_clus()` function. In Table 1, we compare the cluster assignment to the EDD infection status. We observed that the dominant bacteria in each resulting cluster aligns with the findings discussed in Singh et al. (2015).

```
### Obtain the final cluster assignment for the HIV data set.
clusResult <- final_clus( resultList = result_EDD, burn_in = 2500, seed = 1 )
```

Cluster	Healthy	Patient
Cluster 1	4	132
Cluster 2	78	85
Cluster 3	0	1
Cluster 4	0	2
Cluster 5	0	1

Table 1: Distribution of EDD status within the resulting clusters estimated with the proposed model.

Reference

- Dahl, David B, Devin J Johnson, and Peter Müller. 2022. “Search algorithms and loss functions for Bayesian clustering.” *Journal of Computational and Graphical Statistics* 31 (4): 1189–1201.
- Duvallet, Claire, Sean M Gibbons, Thomas Gurry, Rafael A Irizarry, and Eric J Alm. 2017. “Meta-Analysis of Gut Microbiome Studies Identifies Disease-Specific and Shared Responses.” *Nature Communications* 8 (1): 1784.
- Frühwirth-Schnatter, Sylvia, Gertraud Malsiner-Walli, and Bettina Grün. 2021. “Generalized mixtures of finite mixtures and telescoping sampling.” *Bayesian Analysis* 16 (4): 1279–1307.
- Singh, Pallavi, Tracy K Teal, Terence L Marsh, James M Tiedje, Rebekah Mosci, Katherine Jernigan, Angela Zell, et al. 2015. “Intestinal microbial communities associated with acute enteric infections and disease recovery.” *Microbiome* 3: 1–12.